Pocket Packet Sniffer

Software Architecture & Design Specification

WiFi / Ethernet / BLE Portable Packet Sniffing & Dashboard Tool

Document ID	PPS-SADS-1.0
Version	1.0
Standard	IEEE Recommended Practice Format
Date	28 September 2026
Status	Draft for Review — Part 1 Deliverable

1. Introduction

1.1 Purpose

This Software Architecture & Design Specification (SADS) describes the architectural and detailed design of the Pocket Packet Sniffer, translating the requirements in the Software Requirements Specification (PPS-SRS-1.0) into a concrete component structure, interaction design, API design, and security architecture. It is intended for the developers implementing the system and for evaluators assessing design completeness for Part 1.

1.2 Scope

The design covers the on-device edge software stack (ESP32 capture firmware, Raspberry Pi host services, Zeek integration, dashboard) required to satisfy all functional requirements FR-1 through FR-15 and non-functional/security requirements defined in the SRS. It does not cover a future multi-device fleet-management backend, which is out of scope for Part 1.

1.3 Definitions, Acronyms, and Abbreviations

- API – Application Programming Interface
- REST – Representational State Transfer
- WSS – WebSocket Secure
- RBAC – Role-Based Access Control
- STRIDE – Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege (threat modeling framework)

1.4 References

- Software Requirements Specification – Pocket Packet Sniffer (PPS-SRS-1.0).
- Software Test Plan – Pocket Packet Sniffer (PPS-TP-1.0).
- IEEE Std 1016-2009, Standard for Information Technology – Systems Design – Software Design Descriptions.
- Zeek Scripting and Framework documentation, zeek.org.

2. Architectural Design

2.1 Architecture Overview and Pattern

The Pocket Packet Sniffer adopts a layered, pipeline-oriented architecture combined with an edge-local client–server pattern. Traffic flows through four horizontal layers — Capture, Processing, Backend Services, and Presentation — with a cross-cutting Security layer applied at every layer boundary (defense in depth). Within the Processing and Backend layers, an event-driven / publish–subscribe style is used: the Capture layer produces raw packets onto a local queue/stream; Zeek and the Anomaly/Rule Engine consume and transform that stream into structured events; the Backend Services layer publishes state changes (stats, alerts) to the Presentation layer over a WebSocket channel. This hybrid "layered + pipe-and-filter" pattern was chosen because: (1) it cleanly separates hardware-specific capture code from protocol-agnostic analysis logic, easing unit testing; (2) the pipe-and-filter style matches Zeek's own native event-processing model, avoiding an impedance mismatch; and (3) each layer can be independently scaled or replaced (e.g., swapping Zeek for another analysis engine) with minimal ripple effect, satisfying the maintainability requirement NFR-MAINT-1.

2.2 Component Diagram and Description

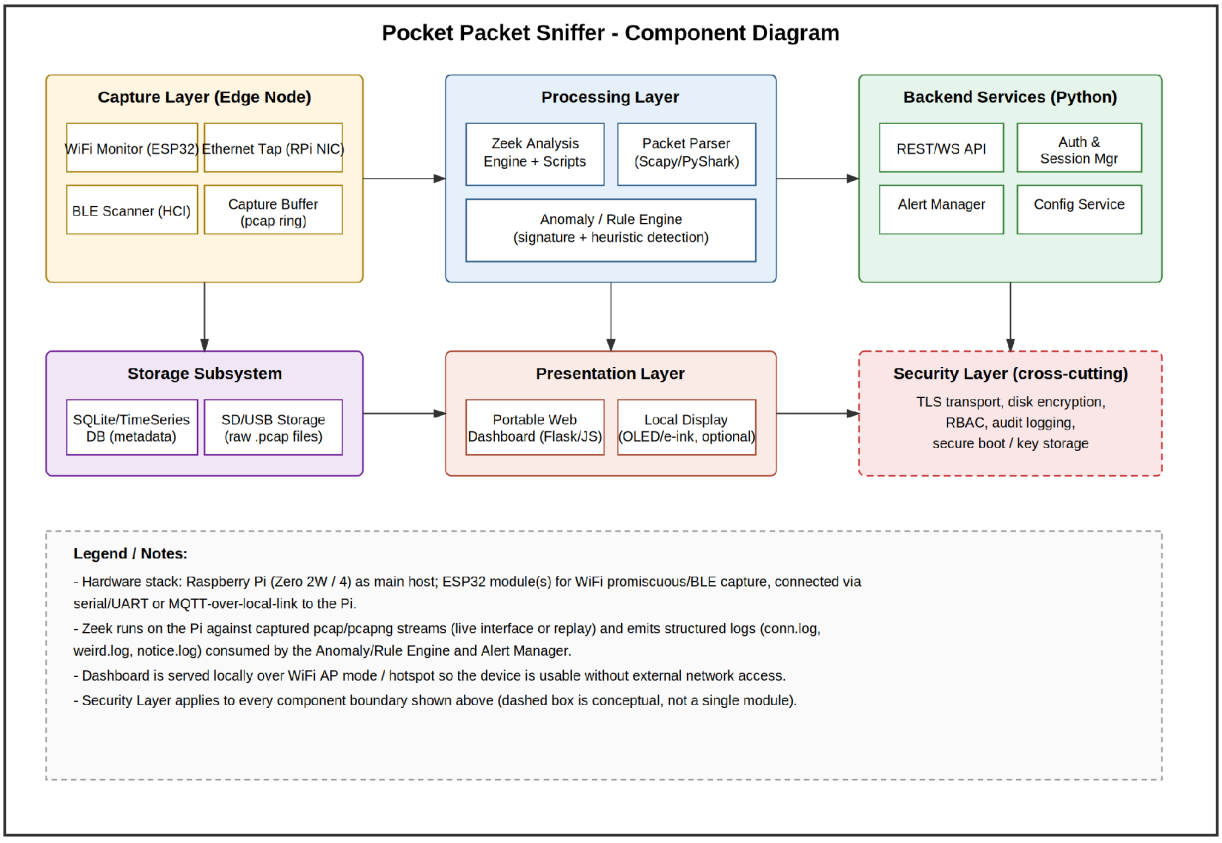


Figure 1. Pocket Packet Sniffer — Component Diagram

2.2.1 Component Descriptions

Component	Responsibility
WiFi Monitor (ESP32)	Places the WiFi radio in monitor mode, captures 802.11 frames on the selected channel, and forwards raw frames to the Capture Buffer.
Ethernet Tap (RPi NIC)	Captures wired frames from the host NIC using a raw socket / libpcap binding.

Component	Responsibility
BLE Scanner (HCI)	Scans BLE advertisement channels via the host Bluetooth HCI interface, applying the configured RSSI threshold.
Capture Buffer (pcap ring)	A bounded in-memory/on-disk ring buffer decoupling capture producers from the Zeek consumer, absorbing short bursts without packet loss.
Zeek Analysis Engine	Consumes the packet stream (live interface or replay) and emits structured logs: conn.log, dns.log, weird.log, notice.log.
Packet Parser (Scapy/PyShark)	Performs supplementary parsing for protocol fields not exposed by default Zeek logs (e.g., raw BLE GATT payloads) and supports .pcap export.
Anomaly/Rule Engine	Evaluates Zeek notices and custom heuristics (ARP spoof, port scan, rogue AP, BLE flood) against configurable thresholds and raises alert events.
REST/WS API	Exposes capture control, statistics, and configuration endpoints to the dashboard; pushes live updates over WebSocket.
Auth & Session Manager	Handles login, session tokens, RBAC checks, and account lockout policy.
Alert Manager	Deduplicates, enriches, persists, and pushes security alerts; records acknowledgments.
Config Service	Centralizes runtime configuration (filters, retention policy, thresholds, user accounts).
SQLite/TimeSeries DB	Stores session metadata, alerts, users, and audit log entries.
SD/USB Storage	Persists raw, encrypted .pcap capture files.
Portable Web Dashboard	Renders live traffic graphs, alert panel, and session controls in-browser.
Local Display (optional)	On-device status display (boot state, IP address, battery).
Security Layer (cross-cutting)	TLS transport, at-rest encryption, RBAC enforcement, and audit logging applied across all layer boundaries (see Section 2.5).

2.3 Architecture Pattern Justification

- Layered separation isolates hardware/driver volatility (ESP32 firmware, WiFi/BLE chipset quirks) from the stable analysis and presentation code, aiding maintainability (NFR-MAINT-1, NFR-MAINT-2).
- Pipe-and-filter data flow (Capture → Zeek → Rule Engine → Alert Manager) directly supports the low-latency requirements NFR-PERF-1 and FR-5/FR-11 by allowing each stage to process independently and asynchronously.
- A bounded ring buffer between Capture and Processing layers provides backpressure handling, supporting the packet-loss bound in NFR-REL-1.
- Local client–server (device-hosted dashboard) supports the offline-first constraint FR-14 without depending on external infrastructure.

2.4 Traceability to Requirements

Architectural Element	SRS Requirement(s) Addressed
Capture Layer (WiFi/Ethernet/BLE modules)	FR-1, FR-2, FR-3, FR-15
Capture Buffer (ring)	NFR-REL-1
Zeek Analysis Engine	FR-5
Anomaly/Rule Engine	FR-10, FR-11
REST/WS API + Dashboard	FR-6, FR-7, FR-9
Packet Parser / Export path	FR-8
Alert Manager	FR-11, FR-12

Architectural Element	SRS Requirement(s) Addressed
Config Service	FR-4, FR-13
Local WiFi AP hosting	FR-14
Auth & Session Manager	SEC-REQ-2, SEC-REQ-3, SEC-REQ-6
Storage encryption (Security Layer)	SEC-REQ-1
TLS termination (Security Layer)	SEC-REQ-4
Audit logging (Security Layer + DB)	SEC-REQ-5

2.5 Security Architecture

The Security Architecture applies defense-in-depth across every component boundary identified in Figure 1, rather than concentrating controls in a single module. It is organized around the security objectives SEC-OBJ-1 (Confidentiality), SEC-OBJ-2 (Access Control), and SEC-OBJ-3 (Integrity & Accountability) from the SRS.

Security Control	Layer / Component	Addresses
AES-256 at-rest encryption of .pcap files and SQLite DB	Storage Subsystem	SEC-REQ-1 / SEC-OBJ-1
TLS 1.2+ on all dashboard/API/WebSocket traffic	REST/WS API, Dashboard	SEC-REQ-4 / SEC-OBJ-1
Credential policy, session tokens, lockout after 5 failures	Auth & Session Manager	SEC-REQ-2, SEC-REQ-6 / SEC-OBJ-2
RBAC (Field Engineer vs. Administrator)	Auth & Session Manager, API layer	SEC-REQ-3 / SEC-OBJ-2
Append-only, hash-chained audit log	Alert Manager, Config Service, DB	SEC-REQ-5 / SEC-OBJ-3
Input validation / bounds checking on all API endpoints	REST/WS API	Mitigates injection & DoS, supports SEC-OBJ-2/3

2.5.1 Threat Model Summary (STRIDE)

Threat Category	Example Threat	Mitigation
Spoofing	Attacker impersonates a legitimate dashboard user	Strong authentication + session tokens (SEC-REQ-2)
Tampering	Attacker modifies stored .pcap evidence	At-rest encryption + hash-chained audit log (SEC-REQ-1, SEC-REQ-5)
Repudiation	User denies performing a config change or export	Audit log records user, action, and timestamp (SEC-REQ-5)
Information Disclosure	Captured traffic intercepted on the local link	TLS 1.2+ on all transport (SEC-REQ-4)
Denial of Service	Malformed API requests crash the backend	Input validation, rate limiting, supervised process restart (NFR-REL-2)
Elevation of Privilege	Field Engineer account performs admin actions	RBAC enforcement at API layer (SEC-REQ-3)

2.6 Deployment View

The entire software stack (Capture, Processing, Backend, Presentation) is deployed on a single physical node — the Raspberry Pi — with the ESP32 acting as an attached peripheral device for WiFi/BLE capture over UART/serial. The

dashboard is served over the Pi's own WiFi access point; no external server or cloud component is required for Part 1 operation, consistent with FR-14.

3. Detailed Design

3.1 UML Sequence Diagrams

Two representative sequence diagrams are provided: (1) the primary capture-session and live-dashboard-update flow, and (2) the anomaly-detection and security-alert flow, which together exercise the majority of the component interactions described in Section 2.2.

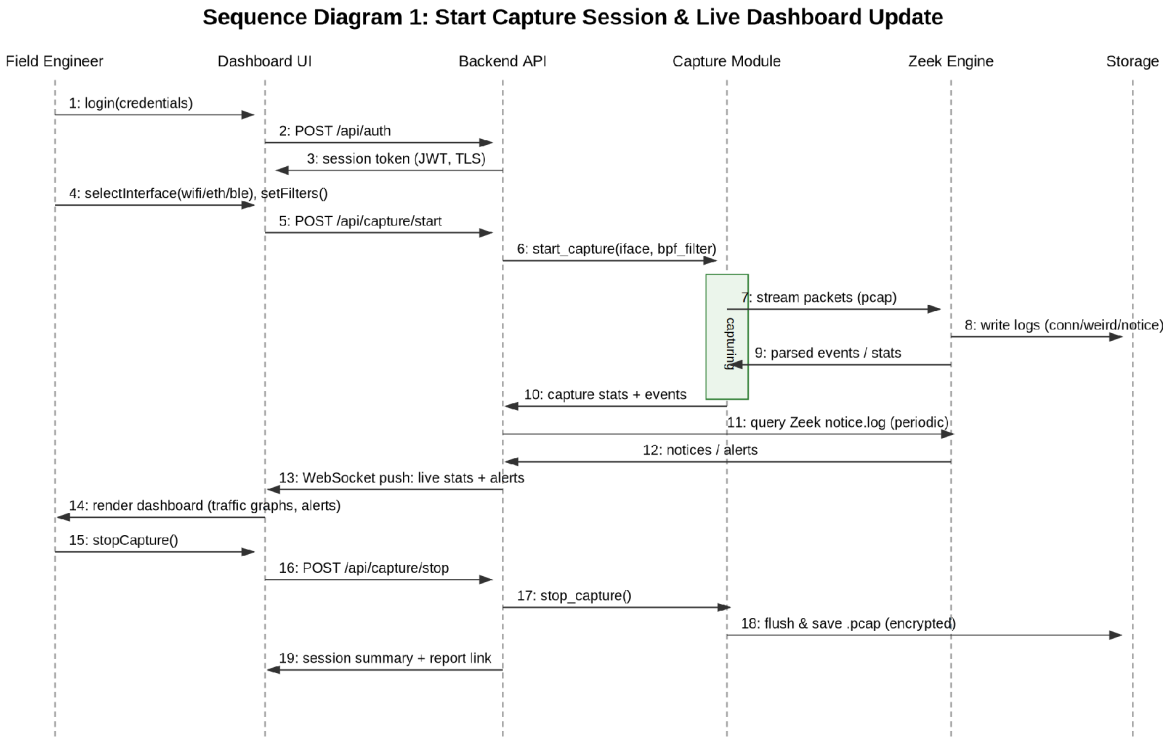


Figure 2. Sequence Diagram 1 — Start Capture Session & Live Dashboard Update

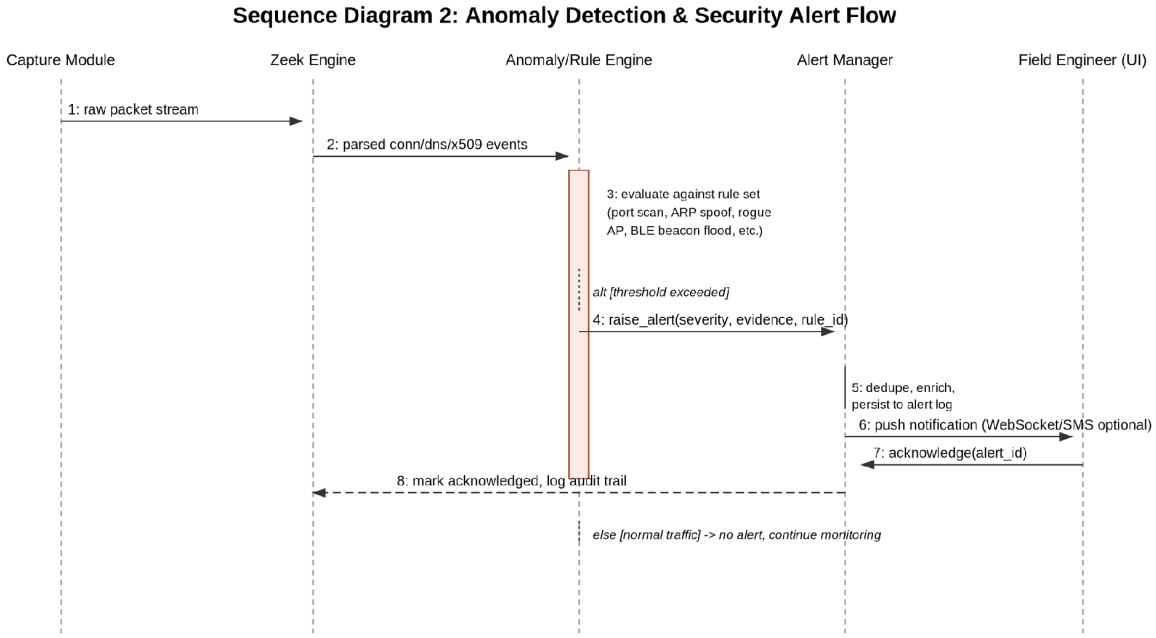


Figure 3. Sequence Diagram 2 — Anomaly Detection & Security Alert Flow

3.2 API Design

The Backend Services layer exposes a REST API for control/configuration and a WebSocket channel for live push updates. All endpoints require a valid session token (SEC-REQ-2) except /api/auth/login, and all Administrator-only endpoints additionally enforce RBAC (SEC-REQ-3). All traffic is served over HTTPS/WSS (SEC-REQ-4).

Method & Endpoint	Description	Auth/Role	Related Req.
POST /api/auth/login	Authenticate user, return session token	Public (rate-limited, lockout after 5 fails)	SEC-REQ-2, SEC-REQ-6
POST /api/auth/logout	Invalidate current session token	Any authenticated user	SEC-REQ-2
GET /api/interfaces	List available capture interfaces (WiFi/Ethernet/BLE)	Field Engineer+	FR-1–FR-3
POST /api/capture/start	Start a capture session with interface, filter, and options	Field Engineer+	FR-4, FR-7, FR-15
POST /api/capture/stop	Stop the active capture session	Field Engineer+	FR-7
GET /api/capture/stats	Retrieve current session statistics snapshot	Field Engineer+	FR-6
WS /api/capture/stream	WebSocket push of live stats and alert events	Field Engineer+	FR-6, FR-11
GET /api/sessions/{id}/export	Download session as .pcap/.pcapng	Field Engineer+	FR-8
GET /api/sessions/{id}/report	Generate and download session summary report	Field Engineer+	FR-9
GET /api/alerts	List active/historical alerts	Field Engineer+	FR-11
POST /api/alerts/{id}/ack	Acknowledge an active alert	Field Engineer+	FR-12, SEC-REQ-5
GET /api/storage	View storage usage and retained sessions	Administrator	FR-13
DELETE /api/sessions/{id}	Delete a stored capture session	Administrator	FR-13
PUT /api/config/retention	Configure retention period	Administrator	FR-13
GET/POST /api/users	List / create user accounts and roles	Administrator	SEC-REQ-3
GET /api/audit-log	Retrieve tamper-evident audit log entries	Administrator	SEC-REQ-5

3.2.1 Sample API Contract — POST /api/capture/start

Field	Detail
Request Body	{ "interface": "wifi" | "ethernet" | "ble", "channel": int (WiFi only), "filter": string (BPF syntax), "rssi_threshold": int (BLE only) }
Success Response	201 Created → { "session_id": string, "started_at": ISO-8601 timestamp }
Error Responses	400 Bad Request (invalid filter/interface), 401 Unauthorized, 409 Conflict (interface already capturing)
Validation Rules	Filter string length ≤ 256 chars, restricted to whitelisted BPF token grammar to prevent command injection; interface must be one of the enumerated values.

3.3 Data Design (Summary)

Core persisted entities in the SQLite/time-series store: Sessions (id, interface, filter, start/end time, packet/byte counts), Alerts (id, session_id, rule_id, severity, evidence, status, ack_by, ack_at), Users (id, username, password_hash, role, lockout_until),

and AuditLog (id, prev_hash, entry_hash, actor, action, timestamp). The AuditLog table's hash-chaining (each entry's hash incorporates the previous entry's hash) directly implements the tamper-evidence requirement SEC-REQ-5.

3.4 Error Handling

- API layer: all endpoints return structured JSON errors { "error_code": string, "message": string } with appropriate HTTP status codes (400/401/403/404/409/500); no internal stack traces are exposed to the client.
- Capture layer: on capture-source failure (e.g., interface disconnected), the Capture Buffer emits a fault event; the Backend marks the session "interrupted" and the dashboard displays a non-blocking warning rather than crashing the session.
- Zeek/Rule Engine: malformed or unparsable frames are logged to weird.log by Zeek and skipped rather than halting the pipeline, preserving throughput for well-formed traffic.
- Backend supervision: a process supervisor (e.g., systemd) automatically restarts a crashed backend service within 30 seconds, satisfying NFR-REL-2; in-flight capture state is checkpointed periodically to the DB to minimize data loss on restart.
- All unhandled exceptions are caught at the top-level request/event handler, logged with a full stack trace to a persistent log file (NFR-MAINT-2), and converted to a safe generic error response to the client.

3.5 Other Design Considerations

- Concurrency: capture producers (WiFi/Ethernet/BLE) run as independent OS processes/threads writing to the shared ring buffer with lock-free or mutex-protected enqueue operations to avoid blocking capture under load (supports FR-15).
- Configurability: capture filters, RSSI thresholds, and anomaly-rule thresholds are externalized to the Config Service rather than hard-coded, easing tuning without code changes.
- Portability: the backend and dashboard are containerizable (Docker) to ease deployment across Raspberry Pi models and future hardware revisions.
- Testability: each layer exposes a mockable interface boundary (e.g., a replay-file input mode for the Capture layer) so that unit and integration tests (Test Plan Section 9) do not require live radio hardware.

4. Design-Level Traceability Matrix

SRS Requirement ID	Architectural Component(s)	Design Artifact
FR-1, FR-2, FR-3	WiFi Monitor, Ethernet Tap, BLE Scanner	Component Diagram (2.2), Sequence Diagram 1 (3.1)
FR-4	Config Service, REST API	API: POST /api/capture/start (3.2.1)
FR-5	Zeek Analysis Engine	Sequence Diagram 1, steps 7–9 (3.1)
FR-6	REST/WS API, Dashboard	API: GET /api/capture/stats, WS /api/capture/stream (3.2)
FR-7	REST API, Capture Buffer	API: POST /api/capture/start|stop (3.2)
FR-8	Packet Parser, Storage	API: GET /api/sessions/{id}/export (3.2)
FR-9	Backend Services	API: GET /api/sessions/{id}/report (3.2)
FR-10, FR-11	Anomaly/Rule Engine, Alert Manager	Sequence Diagram 2 (3.1)
FR-12	Alert Manager	API: POST /api/alerts/{id}/ack (3.2)
FR-13	Config Service, Storage	API: GET /api/storage, DELETE /api/sessions/{id} (3.2)
FR-14	Deployment View	Section 2.6
FR-15	Capture Buffer, concurrency design	Section 3.5 (Concurrency)
SEC-REQ-1	Storage Subsystem (encryption)	Section 2.5, Data Design (3.3)
SEC-REQ-2, SEC-REQ-6	Auth & Session Manager	API: POST /api/auth/login (3.2)
SEC-REQ-3	Auth & Session Manager, API RBAC checks	Section 2.5, API table role column (3.2)
SEC-REQ-4	Security Layer (TLS)	Section 2.5, Communications design
SEC-REQ-5	Alert Manager, AuditLog table	Section 3.3 (hash-chained AuditLog)